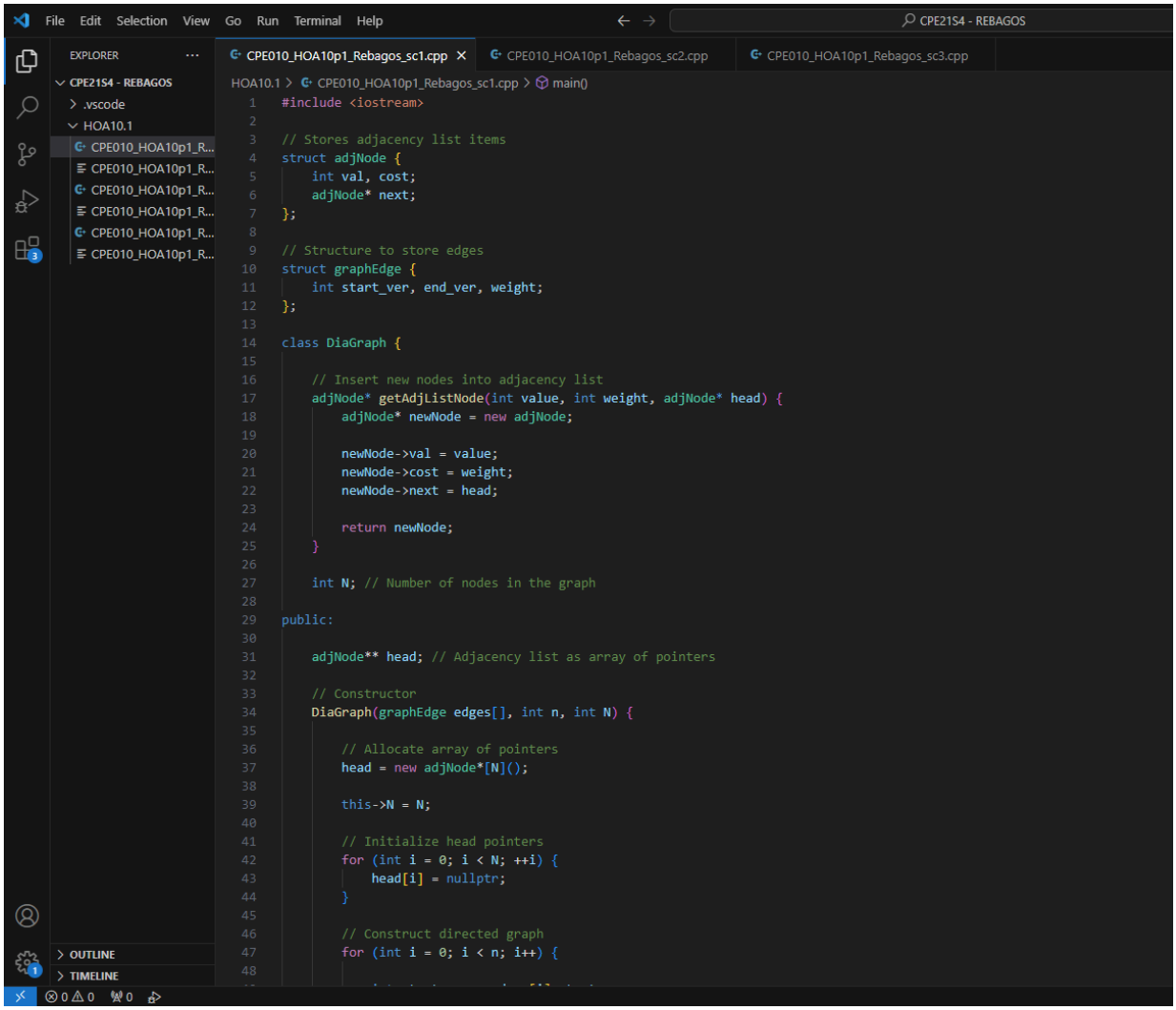
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	02/10/2026	**DATE SUBMITTED:**	02/10/2026
NAME:	Seth Carlo T. Miraflor Kian Joros C. Rebagos	**INSTRUCTOR:**	Engr. Jimlord Quejado

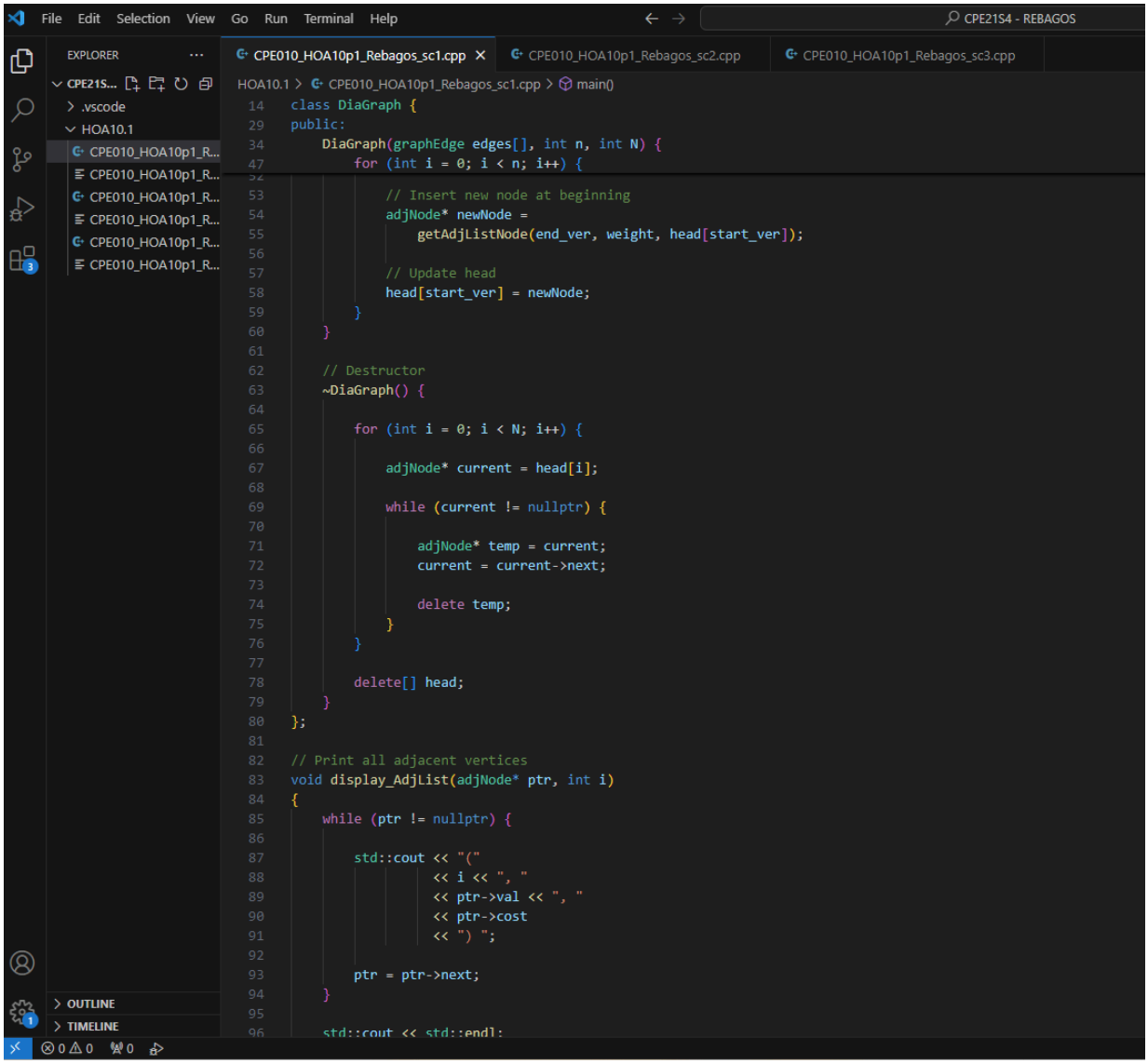
ACTIVITY NO. 10.1 GRAPHS

1. PROCEDURE OUTPUT



COMPUTER ENGINEERING

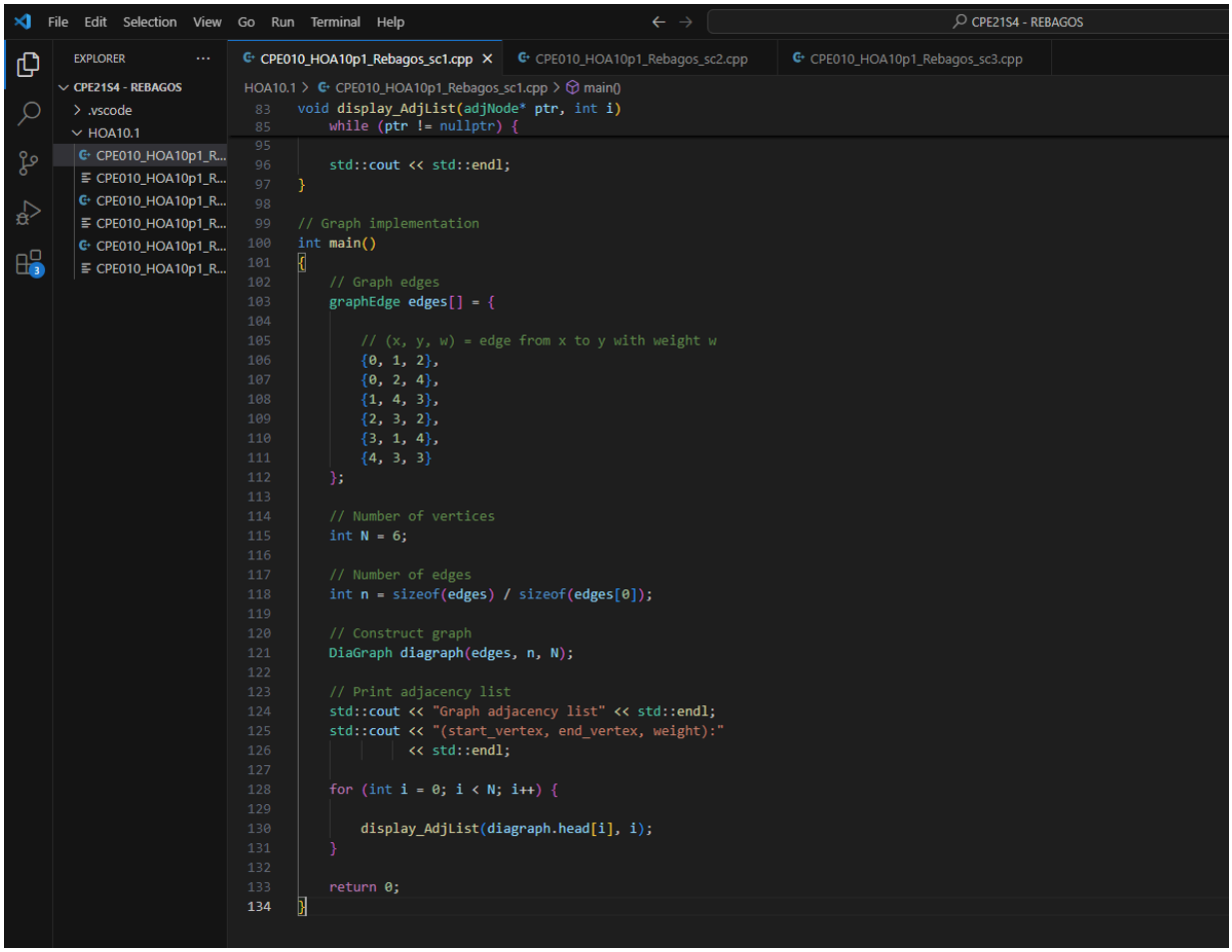
QUEZON CITY

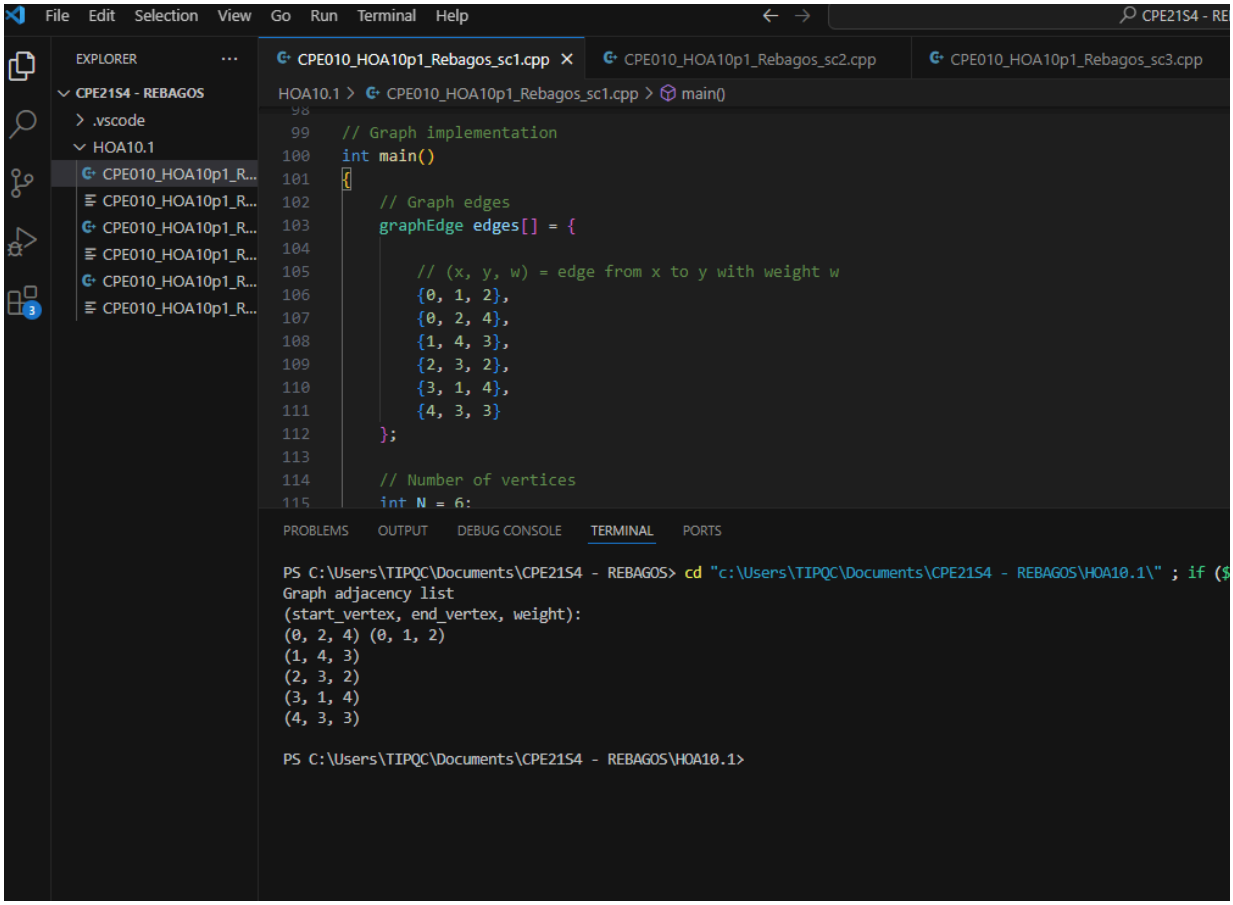



```
14  class DiaGraph {
29  public:
34      DiaGraph(graphEdge edges[], int n, int N) {
47          for (int i = 0; i < n; i++) {
53              // Insert new node at beginning
54              adjNode* newNode =
55                  getAdjListNode(end_ver, weight, head[start_ver]);
56
57              // Update head
58              head[start_ver] = newNode;
59          }
60      }
61
62      // Destructor
63      ~DiaGraph() {
64
65          for (int i = 0; i < N; i++) {
66
67              adjNode* current = head[i];
68
69              while (current != nullptr) {
70
71                  adjNode* temp = current;
72                  current = current->next;
73
74                  delete temp;
75              }
76          }
77
78          delete[] head;
79      }
80  };
81
82  // Print all adjacent vertices
83  void display_AdjList(adjNode* ptr, int i)
84  {
85      while (ptr != nullptr) {
86
87          std::cout << "("
88                    << i << ", "
89                    << ptr->val << ", "
90                    << ptr->cost
91                    << ") ";
92
93          ptr = ptr->next;
94      }
95
96      std::cout << std::endl;
```

COMPUTER ENGINEERING

QUEZON CITY




File Edit Selection View Go Run Terminal Help
CPE21S4 - RE
EXPLORER
CPE21S4 - REBAGOS
.vscode
HOA10.1
CPE010_HOA10p1_R...
CPE010_HOA10p1_R...
CPE010_HOA10p1_R...
CPE010_HOA10p1_R...
CPE010_HOA10p1_R...
CPE010_HOA10p1_R...
CPE010_HOA10p1_Rebagos_sc1.cpp
CPE010_HOA10p1_Rebagos_sc2.cpp
CPE010_HOA10p1_Rebagos_sc3.cpp
HOA10.1 > CPE010_HOA10p1_Rebagos_sc1.cpp > main()
99 // Graph implementation
100 int main()
101 {
102 // Graph edges
103 graphEdge edges[] = {
104
105 // (x, y, w) = edge from x to y with weight w
106 {0, 1, 2},
107 {0, 2, 4},
108 {1, 4, 3},
109 {2, 3, 2},
110 {3, 1, 4},
111 {4, 3, 3}
112 };
113
114 // Number of vertices
115 int N = 6;
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\TIPQC\Documents\CPE21S4 - REBAGOS> cd "c:\Users\TIPQC\Documents\CPE21S4 - REBAGOS\HOA10.1\" ; if ($
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
PS C:\Users\TIPQC\Documents\CPE21S4 - REBAGOS\HOA10.1>

Analysis Procedure for ILO-A

1. Structure
   This code implements a weighted directed graph using an adjacency list. The DiaGraph class manages an array of pointers (head), where each pointer points to a linked list of adjNode structs representing outgoing edges.

2. Graph Construction
- Constructor initializes head array of size N (6 vertices) with nullptr.
- Loops through n (6 edges) and prepends each new node to the front of the starting vertex list in O(1) time.
- Because nodes are prepended to the front, edges for vertex 0 appear in reverse order of input.

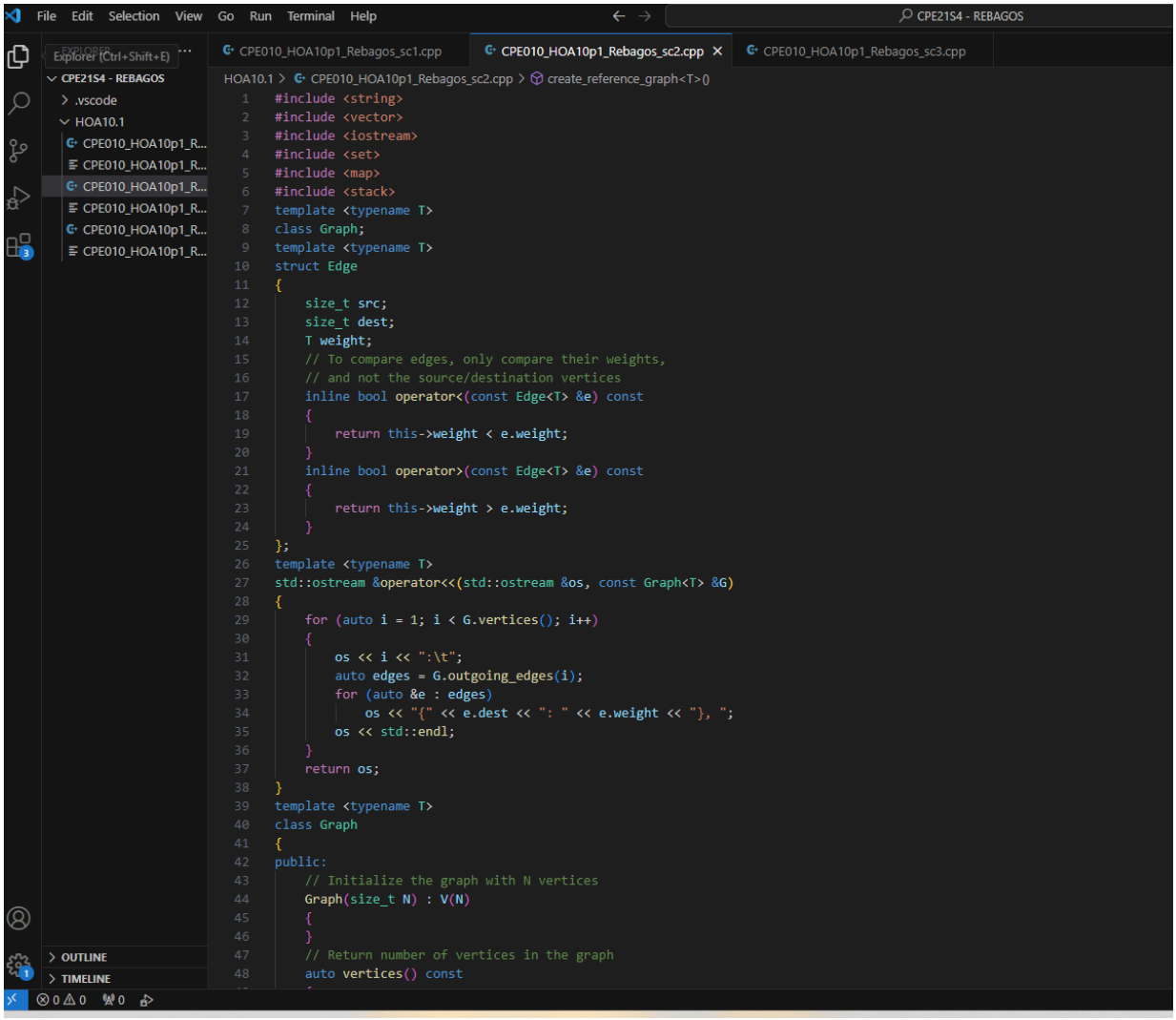
3. Output Trace
   The display_AdjList function prints the outgoing edges for each vertex:
   Vertex 0: (0, 2, 4) (0, 1, 2)
   Vertex 1: (1, 4, 3)
   Vertex 2: (2, 3, 2)
   Vertex 3: (3, 1, 4)
   Vertex 4: (4, 3, 3)
   Vertex 5: (empty)

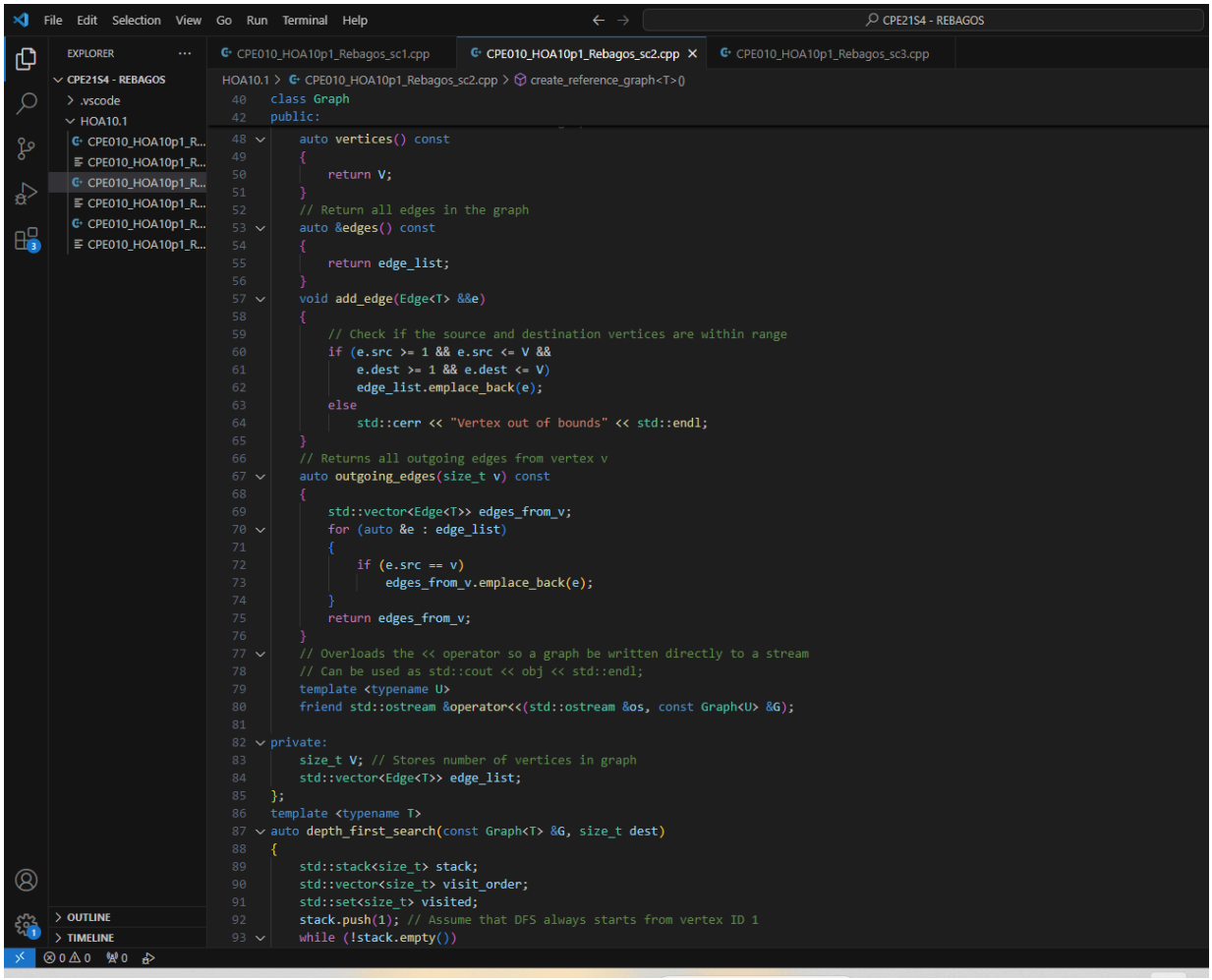
4. Memory Management
   The ~DiaGraph destructor iterates through each linked list, deletes every dynamically allocated adjNode, and then frees the head array using delete[]. No memory leaks occur.

5. Complexity
- Time Complexity: O(V + E) for construction, printing, and cleanup.
- Space Complexity: O(V + E) to store vertices and edge nodes.

COMPUTER ENGINEERING

QUEZON CITY



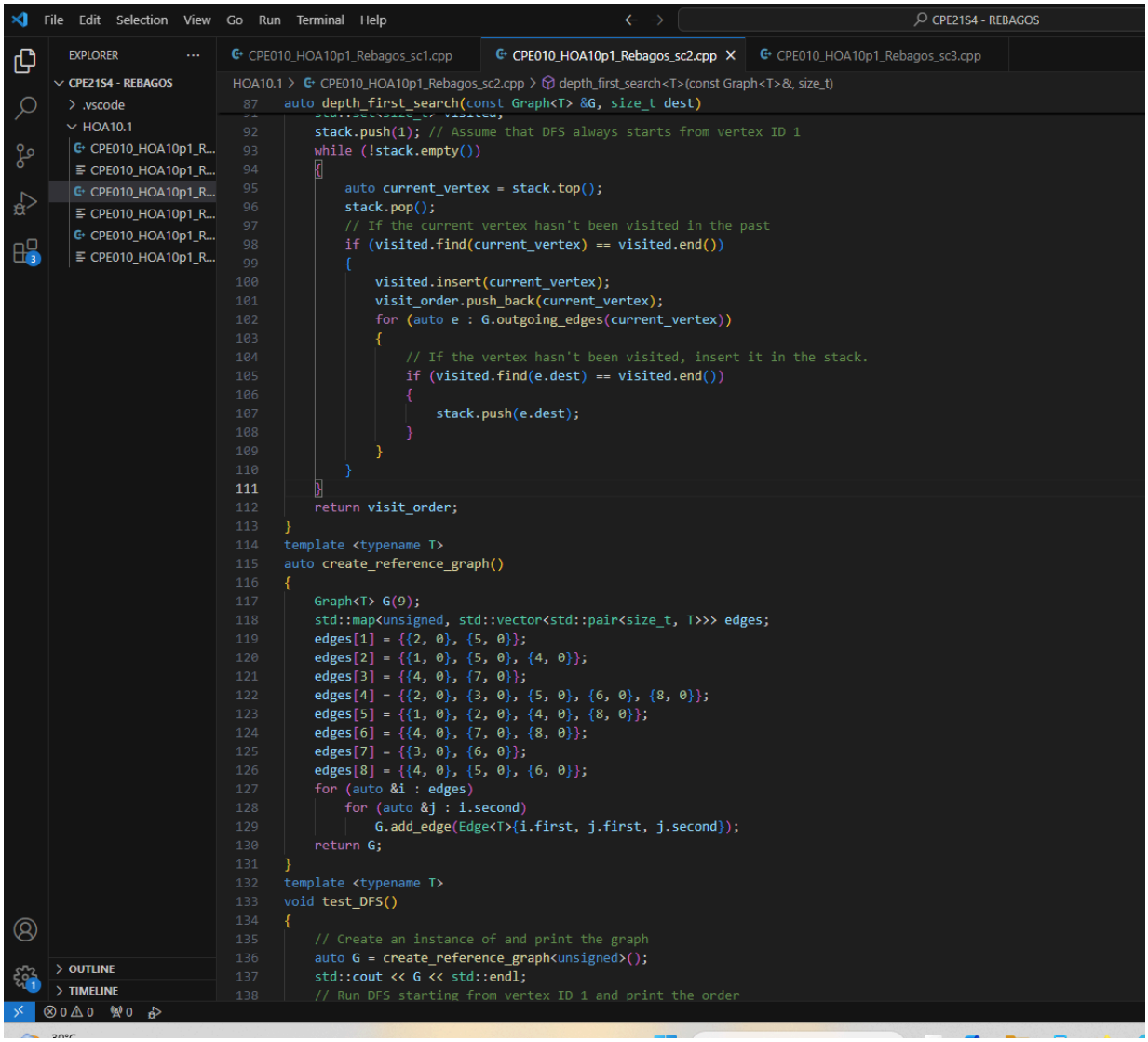



```cpp
class Graph
public:
    auto vertices() const
    {
        return V;
    }
    // Return all edges in the graph
    auto &edges() const
    {
        return edge_list;
    }
    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }
    // Overloads the << operator so a graph be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

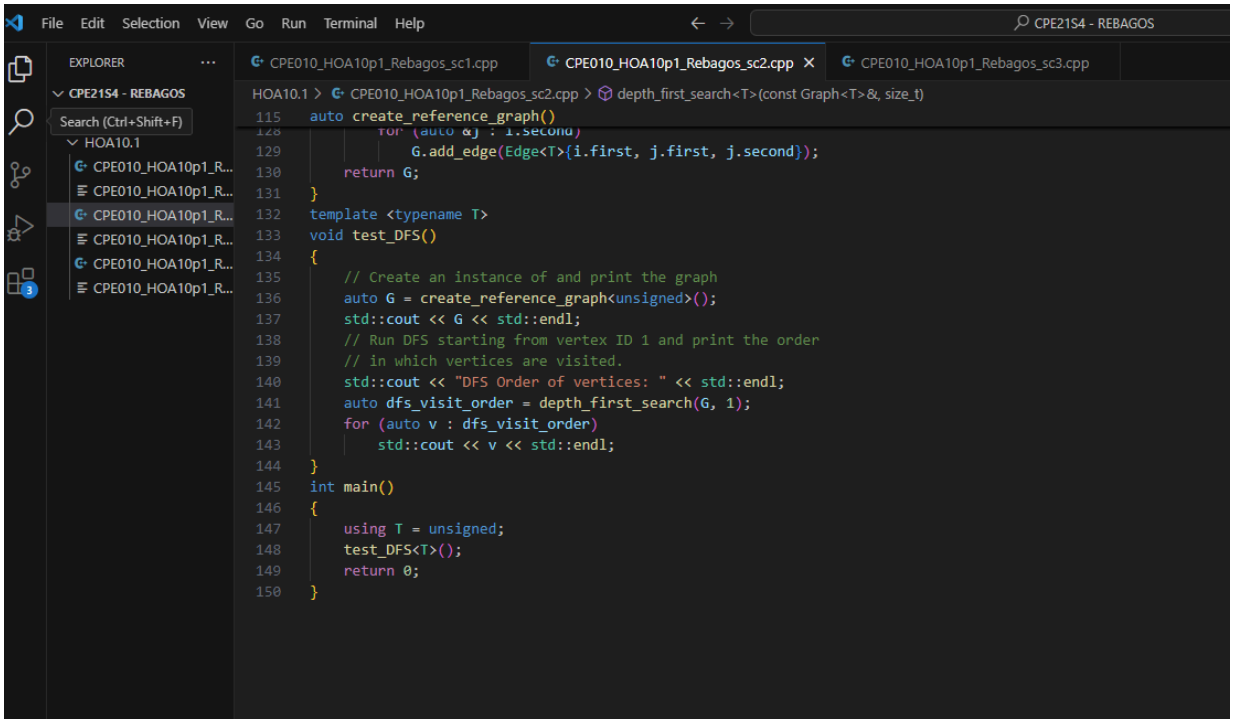
private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty())
```

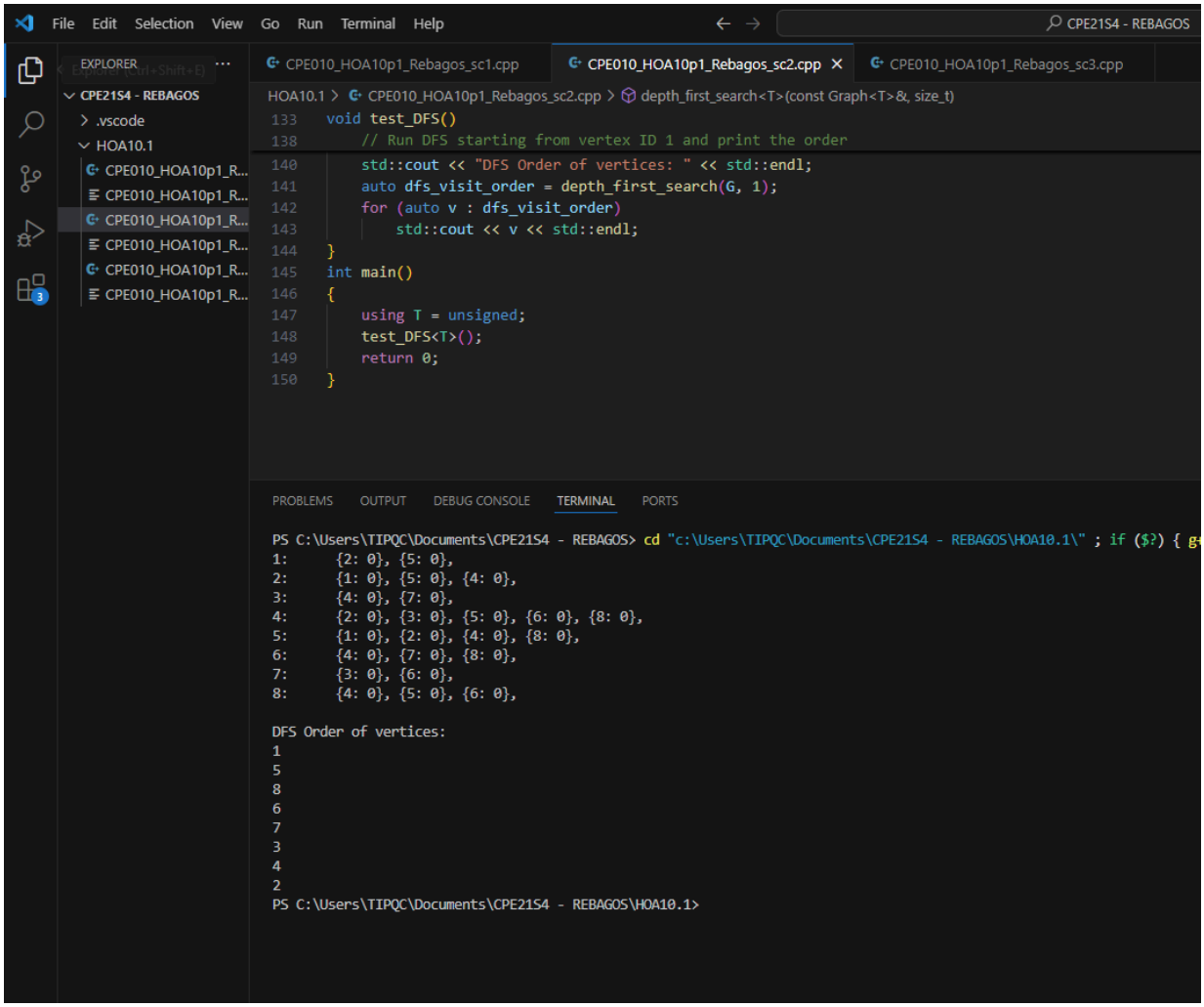
COMPUTER ENGINEERING

QUEZON CITY




```
auto depth_first_search(const Graph<T> &G, size_t dest)
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}
template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
```


File Edit Selection View Go Run Terminal Help
CPE21S4 - REBAGOS
EXPLORER
CPE21S4 - REBAGOS
Search (Ctrl+Shift+F)
HOA10.1
CPE010_HOA10p1_Rebagos_sc1.cpp
CPE010_HOA10p1_Rebagos_sc2.cpp
CPE010_HOA10p1_Rebagos_sc3.cpp
HOA10.1 > CPE010_HOA10p1_Rebagos_sc2.cpp > depth_first_search<T>(const Graph<T>&, size_t)
auto create_reference_graph()
G.add_edge(Edge<T>{i.first, j.first, j.second});
return G;
}
template <typename T>
void test_DFS()
{
// Create an instance of and print the graph
auto G = create_reference_graph<unsigned>();
std::cout << G << std::endl;
// Run DFS starting from vertex ID 1 and print the order
// in which vertices are visited.
std::cout << "DFS Order of vertices: " << std::endl;
auto dfs_visit_order = depth_first_search(G, 1);
for (auto v : dfs_visit_order)
std::cout << v << std::endl;
}
int main()
{
using T = unsigned;
test_DFS<T>();
return 0;
}



Analysis Procedure for ILO-B.1

1. Graph Setup
   The code creates a graph with 9 vertices and stores all 22 edges in a single std::vector. When printing the graph, it looks up outgoing edges for each node:
   1: {2: 0}, {5: 0},
   2: {1: 0}, {5: 0}, {4: 0},
   3: {4: 0}, {7: 0},
   4: {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
   5: {1: 0}, {2: 0}, {4: 0}, {8: 0},
   6: {4: 0}, {7: 0}, {8: 0},
   7: {3: 0}, {6: 0},
   8: {4: 0}, {5: 0}, {6: 0},

2. DFS Traversal Step-by-Step

.It runs an iterative Depth-First Search starting at vertex 1. Because it uses a stack, the last inserted neighbor gets popped and visited first:

- Starts at 1, pushes 2 and 5.
- Visits 5, pushes 2, 4, and 8.
- Visits 8, pushes 4 and 6.
- Visits 6, pushes 4 and 7.
- Visits 7, pushes 3.
- Visits 3, pushes 4.
- Visits 4 (all neighbors already visited).
- Pops already-visited nodes off the stack until reaching 2.
- Visits 2.

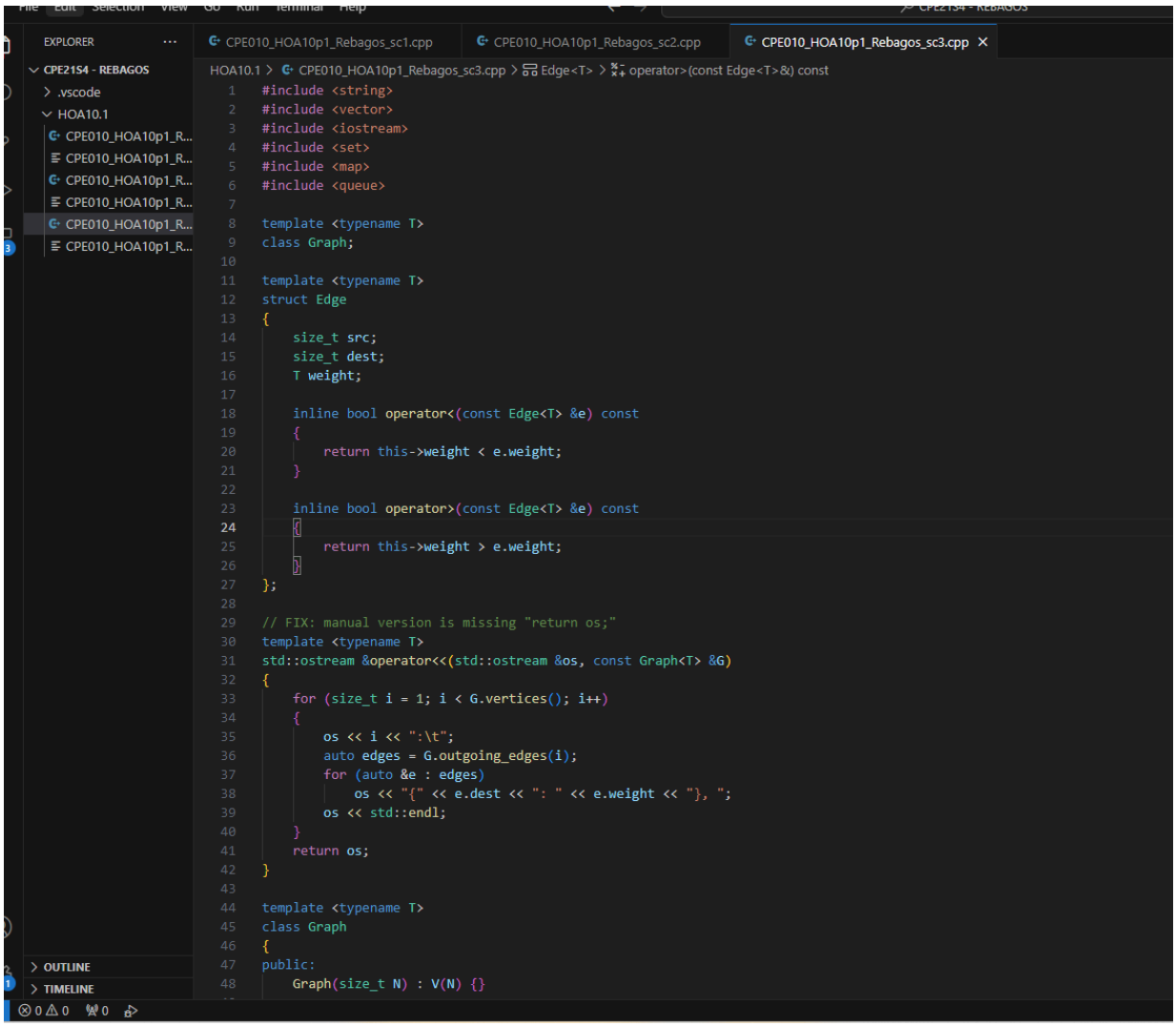
Final Visit Order:
1
5
8
6
7
3
4
2

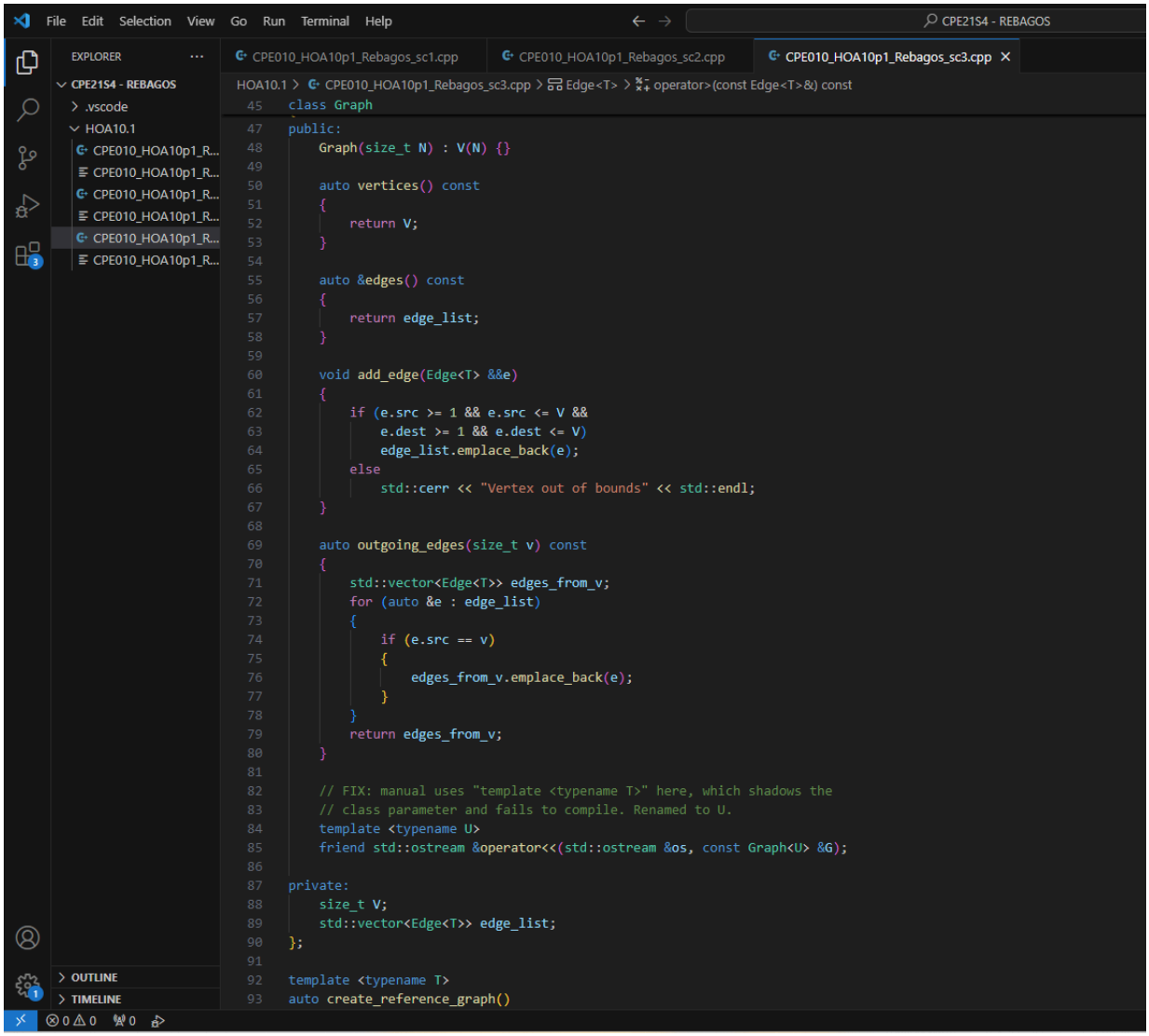
3. Key Takeaways

- Memory: Fully safe. Uses C++ standard containers (vector, set, stack) so everything cleans up automatically.
- Performance: O(V * E) time because outgoing_edges scans the full edge list on every single node visit
- instead of using a direct adjacency lookup.

COMPUTER ENGINEERING

QUEZON CITY






```cpp
class Graph
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

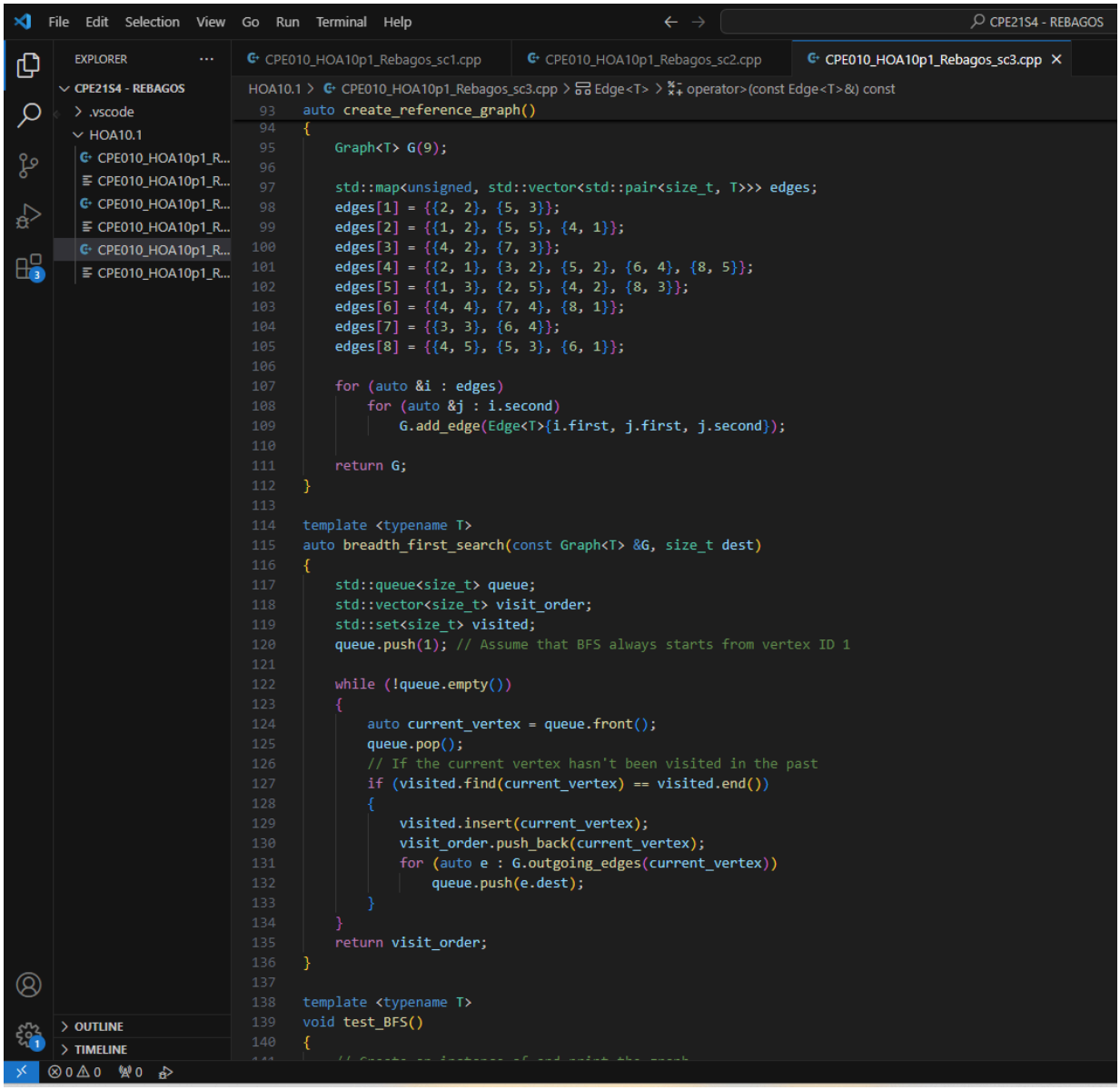
    // FIX: manual uses "template <typename T>" here, which shadows the
    // class parameter and fails to compile. Renamed to U.
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto create_reference_graph()
```

COMPUTER ENGINEERING

QUEZON CITY




```cpp
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

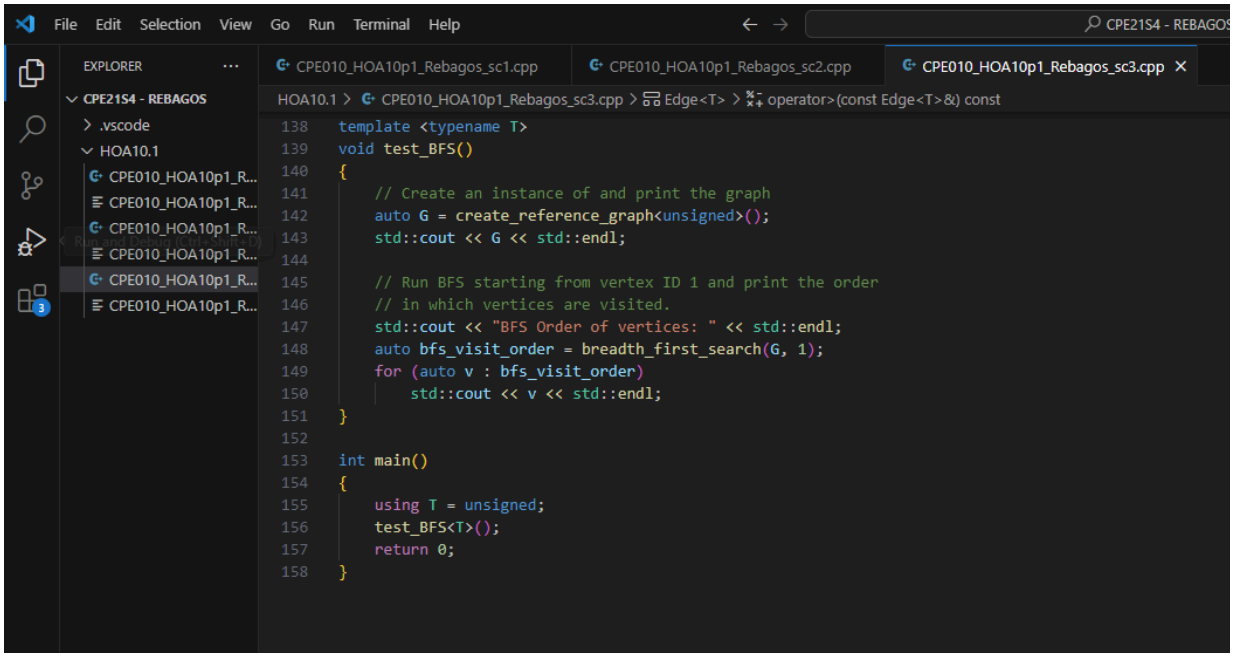
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

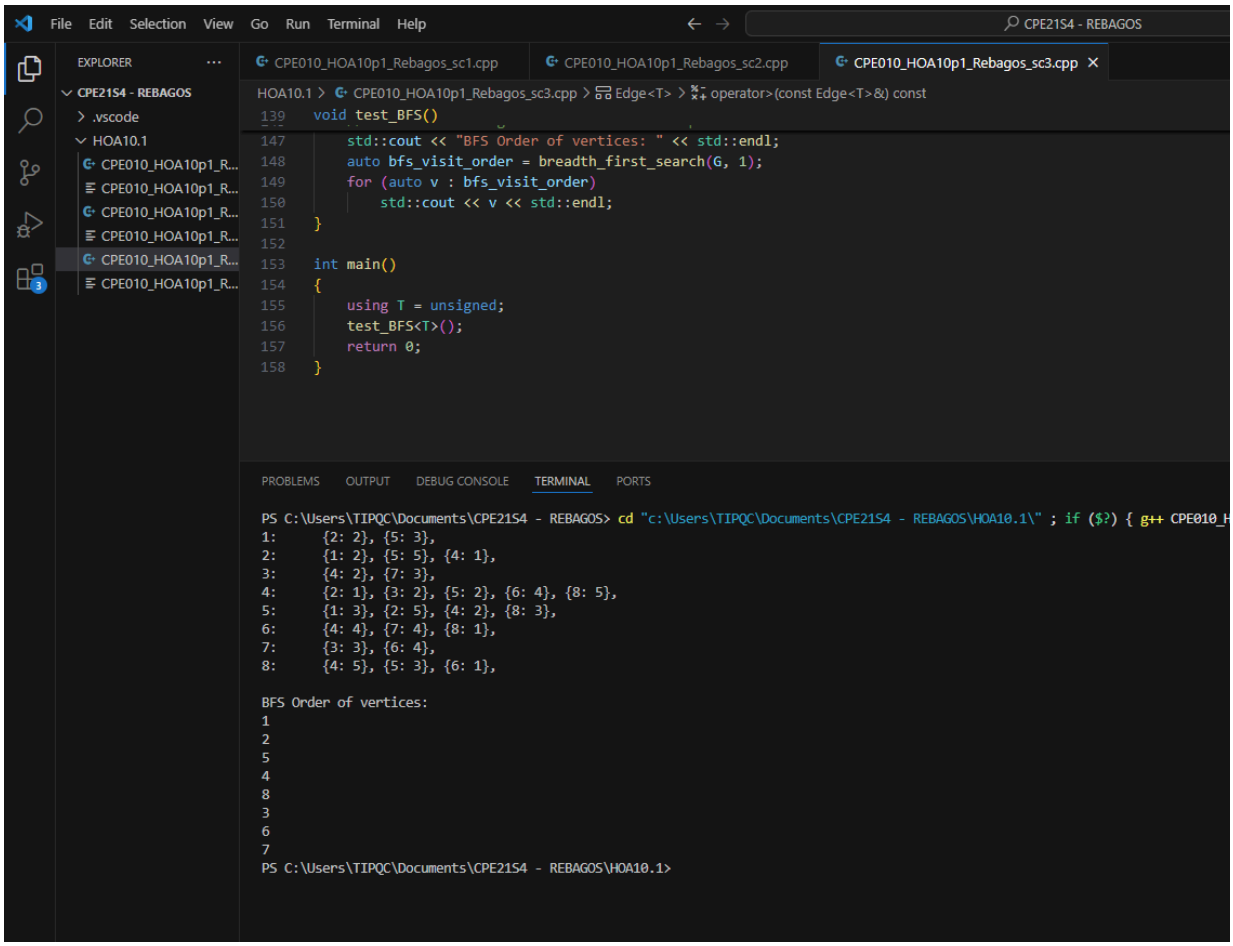
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1

    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}

template <typename T>
void test_BFS()
{
```



```
138  template <typename T>
139  void test_BFS()
140  {
141      // Create an instance of and print the graph
142      auto G = create_reference_graph<unsigned>();
143      std::cout << G << std::endl;
144
145      // Run BFS starting from vertex ID 1 and print the order
146      // in which vertices are visited.
147      std::cout << "BFS Order of vertices: " << std::endl;
148      auto bfs_visit_order = breadth_first_search(G, 1);
149      for (auto v : bfs_visit_order)
150          std::cout << v << std::endl;
151  }
152
153  int main()
154  {
155      using T = unsigned;
156      test_BFS<T>();
157      return 0;
158  }
```



```
PS C:\Users\TIPQC\Documents\CPE21S4 - REBAGOS> cd "c:\Users\TIPQC\Documents\CPE21S4 - REBAGOS\HOA10.1\" ; if ($?) { g++ CPE010_H
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
PS C:\Users\TIPQC\Documents\CPE21S4 - REBAGOS\HOA10.1>
```

Analysis Procedure for ILO-B.2

1. Graph Setup
   Stores 22 weighted edges across 9 vertices in a single vector. Output:
   1: {2: 2}, {5: 3},
   2: {1: 2}, {5: 5}, {4: 1},
   3: {4: 2}, {7: 3},
   4: {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
   5: {1: 3}, {2: 5}, {4: 2}, {8: 3},
   6: {4: 4}, {7: 4}, {8: 1},
   7: {3: 3}, {6: 4},
   8: {4: 5}, {5: 3}, {6: 1},

2. BFS Order (Level by Level)
   Using a queue starting at vertex 1:

- Level 0: 1
- Level 1: 2, 5
- Level 2: 4, 8
- Level 3: 3, 6
- Level 4: 7

Final Visit Order:1, 2, 5, 4, 8, 3, 6, 7

3. Summary

- Memory: Safe automatic cleanup via standard containers.
- Time: O(V * E) due to searching the whole edge vector per node.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- Depth First Search is the right algorithm it goes all the way down one path first then backtracks to the starting point to explore paths like what a person does

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

- Both visit the current node first then it go deep into the next one before backtracking
  DFS: visit node, go to child, repeat, backtrack
  Pre order: Visit node, go left, go right, backtrack

3. In the performed code, what data structure is used to implement the Breadth First Search?

- Queue

4. How many times can a node be visited in the BFS?

- Only once if it already visited a node it never check it again

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.